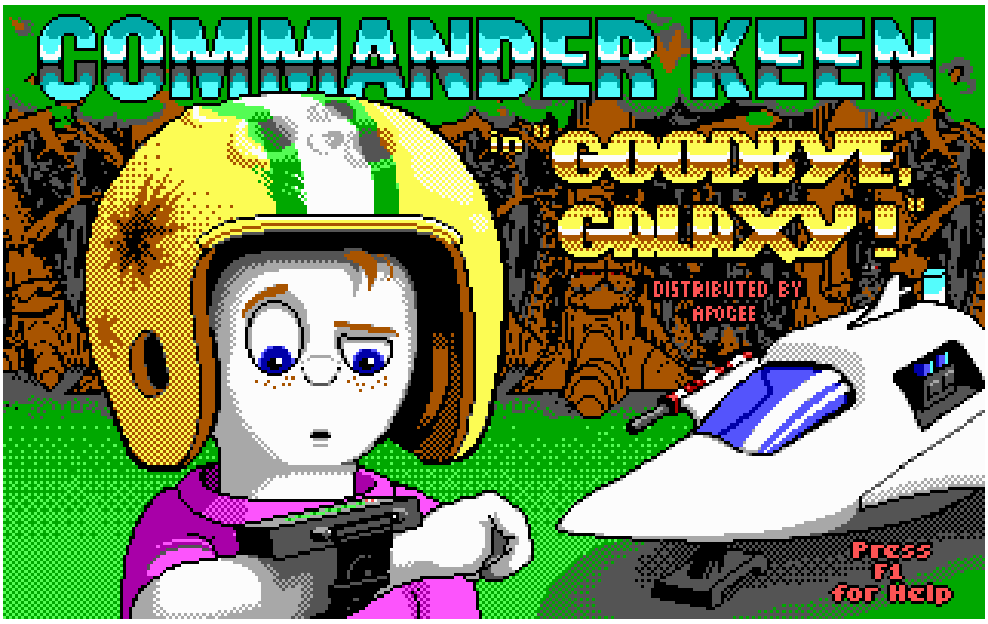
Assignment 4: Commander Keen's PaddleWar!

WebGL without a framework is hard. But this makes it even more rewarding when we create cool stuff!

In class, we learned how to draw multiple objects (rectangles) with different properties (colors and offset). We also made the rectangles move and we added the infamous fish!! In this assignment, we will create a simple but fun video game based on the things we learned. In the game, we will follow in Commander Keen's footsteps and implement the famous PaddleWar! game from his computerwrist device (see below).

By the way, you can play Commander Keen 4 here: `https://www.playdosgames.com/play/commander-keen-4/` and if you navigate to Paddle War in the control panel, you can see the original inspiration for this assignment :). Also, there are many videos of the minigame such as this one: `https://www.youtube.com/watch?v=CJ5ZYxL7XHs`. Overall PaddleWar! is just another Pong clone—a famous game from the Atari era and rebranded many times. There is even a current version for the Nintendo Switch called Proficient Paddles Deluxe: `https://www.youtube.com/watch?v=epsFxm1Jljk`. Legends do live forever :).



Starter code: Please use the code from `https://cs460.org/shortcuts/17` and copy it to your fork as `04/index.html`.

Part 1: Change createRectangle to createPaddle (10 Points)

In class we created the `createRectangle` method to add multiple rectangles to the `OBJECTS` list. Now, we need two paddles to play the game so let's rename the `createRectangle` method to `createPaddle`. Also, we are working on the small display of Keen's computer wrist and need to squeeze the paddles a bit. The following vertices look good so please replace the old code as follows:

```
vertices = new Float32Array( [
                              -0.1,  0.05, 0.0, // V0      // 0
                              -0.1, -0.05, 0.0, // V1, V4  // 1
                               0.1,  0.05, 0.0, // V2, V3  // 2
                               0.1, -0.05, 0.0  // V5      // 3
                             ] );
```



But the paddles do not appear yet, because we will need to create the ball first!

Part 2: Implement createBall (30 Points)

In the starter code, `createBall(color,offset)` is an empty method. Please implement it to render a single point at `0,0,0` using an indexed geometry. Now, please make sure that we render that single point by specifying a `gl_PointSize` of `8` pixels in the vertex shader (make sure it is a float).

So far, so good. However, the ball will not appear since our rectangles are rendered as `gl.TRIANGLES` and here we will need `gl.POINTS`. The best way to address this is to change our geometry representation to include the rendering primitive. Remember, in class we added a vertex count to our representation by changing `[v_buffer,i_buffer,color,offset]` to `[v_buffer,i_buffer,color,offset,6]`. We now need to include another element: the rendering primitive for both `createBall` and `createPaddle`.

After, we will need to change the rendering loop `animate` to load the rendering primitive for each object and modify the `drawElements` call accordingly and our paddles and ball will appear!



Part 3: Add interaction (20 Points)!

But nothing moves yet! For this part, please modify the last method in the code: the `window.onkeydown` callback. Please implement movement of the Keen's paddle—the one in the bottom—when you press `ArrowLeft` or `ArrowRight`. Make sure that the paddle does not leave the viewport and use the `STEP` variable to modify the offset accordingly. After this, you should be able to control the paddle.

Part 4: Study updateBall() and call it! (10 Points)

The starter code includes the `updateBall` method. Please study the code and be ready to explain it when submitting the form. Also, the method needs to be called every frame. Please modify `animate` to call it at the proper time.



With these changes, you can control your paddle while the ball moves. Should be an easy win for the Commander at this time :).

Part 5: Study updateComputer() and call it! (10 Points)

The starter code also includes the `updateComputer` method. Please study the code and be ready to explain it during form submission. We also need to call this one in the `animate` loop to kickstart our advanced artificial intelligence :). Now, the computer will join the game and winning is not that easy anymore!



Part 6: Study resetBall() and score() (10 Points)

There are two more methods to study and to explain: `resetBall` and `score`.

Part 7: Submit the Form! (10 Points)

Finally, please push everything to your fork and double-check Github Pages! As usual, please send a pull request and then fill in the form at `https://cs460.org/assignments/04/` but don't submit it yet if you plan on doing the bonus (aka. the fun part!).

Bonus (Total 33 points):

Bonus Part 1: Add Sound Effects! (5 points)

Add some cool sounds for instance when the ball is reflected and some background music or some effects when you win or loose. You can use the `<audio>` tags for that.

Bonus Part 2: Speed up! (8 points)

The game is relatively boring right now - make the ball move faster with time :).

Bonus Part 3: Shrinking/Growing Paddles! (9 points)

Changing the paddle size could also add some entertainment - be creative :).

Bonus Part 4: Advanced Reflection! (11 points)

The original PaddleWar! allows special moves with the corner of the paddle to either accelerate the ball or change direction. Think about it and add it for the ultimate experience!!

SUPERBONUS: Up to 33 extra points for the most creative solutions!!
Feel free to customize any gameplay or visuals!

